Vellore University OfTechnology

OPENSOURCESOFTWARE (LINUXADMINISTRATION)

Course Code: CSE0002

THE OPEN SOURCE AUDIT (*A Capstone Project Report)*

Report Topic:

"Case Study on Python Programming Language"

Submitted By:
Name : Simmy Singroul
Registration Number: 24BAI10202
Slot: A13

Date of Submission: 31/03/2026

PART – A

A1 — The ProblemPythonWasCreated to Solve

Background of Programming Before Python In the early stages of software development, programming languages such as C and C++ were widely used and formed the foundation of many important systems. These languages were extremely powerful and gave programmers a high level of control over how software interacted with computer hardware. However, this power came with a trade-off. Writing programs often involved dealing with complex syntax, strict formatting rules, and low-level details such as manual memory management. Even small mistakes could lead to errors that were difficult to detect and fix.

Because of this complexity, programming was not easily accessible to everyone. Beginners often found it challenging to understand how programs worked, and developing software required significant time and effort. While these languages were highly effective for system-level programming, they were not well-suited for rapid development or experimentation. As computing began expanding into new domains, there was a clear need for a programming language that was easier to use while still being flexible and powerful.

The Idea Behind Creating Python During the late 1980s, Guido van Rossum was working at the Centrum Wiskunde & Informatica (CWI) in the Netherlands. At that time, he had contributed to the development of a programming language called ABC. ABC was designed to make programming simpler and more user-friendly compared to traditional languages. Although it introduced several useful concepts, it did not gain widespread adoption because it lacked extensibility and had limited integration with other systems. Despite these limitations, the experience played an important role in shaping Guido's vision. He believed that it should be possible to design a language that combined simplicity with real-world usability. Instead of forcing programmers to focus on complex syntax and technical details, the language should allow them to concentrate on solving problems more effectively.

With this idea in mind, Python was developed and first released in 1991. Its primary goal was to create a language that was easy to read, easy to write, and versatile enough to be used across different types of applications.

Addressing the Challenges in Software Development Python was designed to address several key challenges faced by programmers. One of its most important features was readability. The language introduced a clean and structured syntax that made code easier to understand and maintain. This improved collaboration, as developers could more easily read and work with each other's code.

Another major improvement was the speed of development. Traditional programming often required multiple steps such as compiling and linking before execution. Python simplified this process by being

an interpreted language, allowing programs to run directly. This made testing and experimentation much faster, which is especially useful in modern development environments.

Python also reduced the difficulty faced by beginners. Its intuitive structure and straightforward syntax lowered the learning curve, making programming more accessible to a wider audience. This is one of the key reasons why Python is now widely used in education and is often recommended as a first programming language.

The Influence of Open Source Philosophy An important factor in Python's success is its open-source nature. Python was released in a way that allows anyone to use, modify, and distribute it freely. This encouraged collaboration among developers from different parts of the world and helped the language evolve over time. Because the source code is publicly available, developers can contribute improvements, fix bugs, and introduce new features. This community-driven development model has played a significant role in making Python stable, reliable, and widely adopted. The open-source philosophy behind Python reflects the idea that software improves through shared knowledge and collective effort rather than being controlled by a single entity.

Impact and Growth of Python Over time, Python has grown into one of the most widely used programming languages. It is applied in various fields such as web development, automation, scientific computing, data analysis, and artificial intelligence. Many well-known technology companies use Python for building and maintaining their systems.

The language's versatility allows it to adapt to different domains. Its simplicity helps beginners get started easily, while its powerful libraries and frameworks enable professionals to develop complex applications. This balance between ease of use and capability has made Python an essential tool in modern computing**.**

A2 — Software Licensing in Python

Nature of Open Source Licensing Software licensing acts as the legal framework that defines how a software system can be used, modified, and shared. In traditional proprietary models, software is usually distributed in compiled form, and users do not have access to the source code. They are also restricted by strict agreements that limit copying, modification, and redistribution. This approach keeps control centralized within a single organization and often prioritizes commercial protection over collaboration. In contrast, open-source licensing follows a different philosophy. Instead of restricting users, it provides them with clearly defined freedoms while still protecting the rights of the original creators. These licenses allow users to study how the software works, modify it according to their needs, and share it

with others. At the same time, they include certain conditions, such as retaining copyright notices, to ensure proper attribution.

This balance between freedom and responsibility is what makes open-source ecosystems sustainable.

Without licensing, there would be no clear legal structure governing how the software can be used or distributed, which would create confusion and limit adoption. Therefore, licensing is not just a legal requirement but a fundamental component that enables open-source software to grow and function effectively.

The Python Software Foundation License and User Freedoms Python is distributed under the Python Software Foundation (PSF) License, which is considered a permissive open-source license. This means that it places minimal restrictions on how the software can be used and integrated into other systems. Under the PSF License, Python can be used freely for personal, academic, and commercial purposes without any licensing fees. This has played a major role in its widespread adoption across different fields such as education, research, and industry. Organizations can use Python in their products without needing special permissions, which reduces legal barriers.

Another important feature of the PSF License is the freedom to modify and redistribute the software.

Since the full source code is publicly available, developers can study its internal structure, make changes, and even distribute modified versions. The only key requirement is that the original copyright notice and license terms must be retained. This ensures that the original developers are credited, even when the software is extended or reused. These permissions align closely with the core principles defined by the Free Software Foundation, often referred to as the four freedoms of free software. These include the freedom to use, study, modify, and redistribute software. Python satisfies all of these conditions, making it a strong example of an open-source system.

At the same time, because the PSF License is permissive, organizations are not required to release their modifications publicly. This means Python can be used within proprietary systems without forcing companies to disclose their internal changes. This flexibility is one of the key reasons why Python is widely adopted in both open-source and commercial environments.

Comparison with Other Licenses and Practical Implications The nature of the PSF License becomes clearer when compared with other popular open-source licenses such as the GNU General Public License and the MIT License.

The GNU General Public License (GPL) follows a copyleft model, which requires that any modified or derivative work must also be distributed under the same license. This ensures that all versions remain open-source and that improvements are shared with the community. While this strengthens openness, it can discourage use in commercial environments where companies want to keep their modifications private. On the other hand, the MIT License is one of the most permissive licenses available. It allows users to use, modify, and distribute software with very few restrictions. The PSF License is quite similar in spirit, as it also prioritizes flexibility and ease of integration. However, unlike copyleft licenses, permissive

licenses rely more on voluntary contributions rather than enforcing openness through legal requirements.

A commonly discussed concept in open-source is the difference between “free as in cost” and “free as in freedom.” Python satisfies both interpretations. It is available at no cost, but more importantly, it gives users the freedom to control how the software is used and adapted. From my understanding, this freedom is far more valuable than simply being free of cost, as it enables innovation and reduces dependency on specific vendors. The licensing model used by Python has played a major role in its success. Its permissive nature encourages both individuals and organizations to adopt it without worrying about strict legal obligations. At the same time, the requirement to retain attribution ensures that the original work is respected. This balance between openness and flexibility allows Python to function effectively as both a community-driven project and a widely used industry tool.

A3 — Ethics of Open Source Software

The Question of Universal Open Source Adoption The idea that all software should be open source is often presented as an ideal within the technology community, as it promotes transparency, collaboration, and shared innovation. Open-source software allows users to study, modify, and distribute code, which encourages collective problem-solving and reduces dependency on a single organization. This approach can accelerate technological progress, as improvements made by one developer can benefit the entire community. It also enhances security in certain contexts, as publicly available code can be examined and audited by multiple contributors, potentially leading to faster identification and resolution of vulnerabilities. However, the argument for universal adoption of open-source software is not without limitations. Not all software systems can function effectively under a fully open model, particularly in cases where proprietary innovation, competitive advantage, or sensitive data handling is involved. Organizations that invest significant resources in research and development may rely on proprietary systems to protect intellectual property and ensure financial sustainability. Additionally, maintaining large-scale open-source projects requires continuous contributions, coordination, and governance, which may not always be feasible without structured funding or organizational support. Therefore, while open source offers clear advantages, a complete replacement of proprietary software is neither practical nor universally desirable.

Commercial Use of Open Source and Ethical Considerations The relationship between open-source software and commercial enterprises raises important ethical questions. Many large organizations build profitable products and services on top of open-source technologies. For example, companies rely on open-source operating systems, programming languages, and frameworks as the foundation of their infrastructure. This practice is legally permitted under most open-source licenses, particularly permissive ones such as the PSF License, which do not require organizations to disclose modifications or contribute back to the community.

From one perspective, this model is beneficial, as it demonstrates the real-world value of open-source software and contributes to its widespread adoption. Companies often invest in improving performance, security, and scalability, which can indirectly benefit the broader ecosystem. In some cases, organizations actively contribute back to open-source projects by releasing tools, patches, or entire frameworks.

At the same time, concerns arise regarding the balance between community contribution and commercial gain. When companies generate significant profits using open-source software without sharing their improvements, it can create an imbalance where community-driven efforts are not equally rewarded. This raises questions about fairness and sustainability, particularly for projects that rely heavily on voluntary contributions. The ethical challenge lies in determining whether organizations have a moral responsibility, beyond legal obligations, to contribute back to the open-source community that supports their operations.

Open Source and the Nature of Innovation The concept of "standing on the shoulders of giants" is closely associated with open-source software,
as it reflects the idea that new developments are built upon existing knowledge and contributions. Open-source systems provide a foundation that allows developers to focus on innovation rather than reinventing basic components. This accelerates the development process and enables more rapid technological advancement across various fields. At the same time, there is an ongoing debate about whether open source enhances or limits originality.
On one hand, the availability of shared resources fosters creativity by allowing developers to experiment, modify, and extend existing systems. It reduces barriers to entry and encourages participation from a diverse range of contributors. On the other hand, reliance on existing frameworks and tools may lead to incremental improvements rather than entirely new approaches, potentially reducing the incentive to develop fundamentally original solutions.
Despite this concern, open source generally acts as an enabler rather than a limitation to innovation. By providing a collaborative environment and reducing duplication of effort, it allows developers to focus on solving more complex and meaningful problems. The balance between shared knowledge and original contribution remains an important aspect of the open-source ecosystem, shaping how technology evolves over time.

Part – B:

Installation and System Integration of Python

Python is widely recognized as a core component of modern Linux systems and is often included as a default installation in many distributions. Unlike standalone applications that require separate installation and configuration, Python is deeply integrated into the operating system environment due to its role in system utilities and administrative tools. Many Linux distributions rely on Python scripts for package management, system configuration, and automation tasks, which makes its presence essential rather than optional. This level of integration reflects the broader importance of Python within the open-source ecosystem, where it serves both as a development tool and as a supporting component for system-level operations.

The installation of Python in a Linux environment is typically handled through package management systems such **as apt, yum, or dnf,** depending on the distribution being used. These package managers simplify the process by automatically resolving dependencies and ensuring compatibility with existing system components. In Debian-based systems, Python can be installed or verified using commands such as **sudo apt install python3**, which either installs the interpreter or confirms that the latest version is already present.

```
anshika@LAPTOP-BLC9HHCI:/mnt/c/Windows/System32$ sudo apt install python3
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
python3 is already the newest version (3.12.3-0ubuntu2.1).
python3 set to manually installed.
0 upgraded, 0 newly installed, 0 to remove and 63 not upgraded.
anshika@LAPTOP-BLC9HHCI:/mnt/c/Windows/System32$ python3 --version
Python 3.12.3
anshika@LAPTOP-BLC9HHCI:/mnt/c/Windows/System32$
```

This approach highlights the efficiency of Linux package management, where software installation is centralized and standardized rather than requiring manual configuration.

An important aspect of Python's presence in Linux systems is the coexistence of multiple versions. In many cases, a default version of Python is reserved for system-level operations, while a separate version may be used for user-level development. This separation ensures that critical system processes remain stable even when users install or modify different versions for their own applications. The ability to maintain multiple versions without conflict demonstrates the flexibility of Linux environments in managing software dependencies.

Verification of Python's installation is typically performed using commands such as **python3 --version**, which provides information about the installed version of the interpreter. Additionally, commands like which python3 can be used to identify the exact location of the Python executable within the system.

```
anshika@LAPTOP-BLC9HHCI:/mnt/c/Windows/System32$ which python3
/usr/bin/python3
anshika@LAPTOP-BLC9HHCI:/mnt/c/Windows/System32$ ls /usr/bin/python3
/usr/bin/python3
anshika@LAPTOP-BLC9HHCI:/mnt/c/Windows/System32$
```

These commands not only confirm the presence of Python but also provide insight into how it is structured within the Linux file system. Such verification steps are essential in demonstrating that the software is correctly installed and operational.

File System Structure, Users, and Permissions

File System Organization of Python in Linux In a Linux environment, software is not confined to a single directory but is distributed across a
structured file system based on functionality and usage. Python follows this standard organization, with its executable, libraries, and supporting files placed in different system directories. The primary executable for Python is typically located in directories such as /usr/bin/python3, which contains system-wide binaries accessible to all users. The standard libraries associated with Python are stored in directories like **/usr/lib/python3**, where core modules required for Python's functionality are maintained.

```
anshika@LAPTOP-BLC9HHCI:/mnt/c/Windows/System32$ ls /usr/bin/python3
/usr/bin/python3
anshika@LAPTOP-BLC9HHCI:/mnt/c/Windows/System32$ ls -l /usr/bin/python3
lrwxrwxrwx 1 root root 10 Nov 12 12:15 /usr/bin/python3 -> python3.12
anshika@LAPTOP-BLC9HHCI:/mnt/c/Windows/System32$
```

This separation of components reflects the modular design of Linux systems, where binaries, libraries, and configuration files are organized independently to ensure maintainability and clarity. Additional packages installed by users through tools such as pip are often stored in user-specific directories, preventing interference with system-level files. This structured distribution not only improves system stability but also allows multiple applications and users to interact with Python without conflicts. The file system layout therefore plays a critical role in ensuring that Python operates efficiently as both a system utility and a development environment.

User Environment and Access Control

```
anshika@LAPTOP-BLC9HHCI:/mnt/c/Windows/System32$ whoami
anshika
anshika@LAPTOP-BLC9HHCI:/mnt/c/Windows/System32$
```

Linux is a multi-user operating system, and access to files and resources is controlled through a well-defined permission model. Python operates within this environment by inheriting the permissions of the user executing it. This means that any Python script can only access files and perform operations that the current user is authorized to execute. As a result, Python does not function as an independent service with elevated privileges but instead adheres strictly to the user-level security framework of the operating system.

Administrative actions, such as installing system-wide Python packages or modifying core directories, require elevated privileges, typically obtained using commands like sudo. In contrast, regular users can install packages within their own directories without affecting the global system configuration. This distinction ensures that critical system components remain protected from unintended modifications while still allowing flexibility for individual users. The user-based access control model enhances security and stability, particularly in environments where multiple users share the same system.

Permission Mechanisms and Their Practical Implications The Linux permission system is based on three primary categories: read, write, and execute
permissions, which are assigned to the owner, group, and others. These permissions determine how files and directories can be accessed or modified. In the context of Python, execute permission is particularly important, as it allows scripts to be run as programs. For example, a Python script can be made executable using commands such as chmod +x script.py, after which it can be executed directly from the terminal.

```
anshika@LAPTOP-BLC9HHCI:/mnt/c/Windows/System32$ cd ~
anshika@LAPTOP-BLC9HHCI:~$ pwd
/home/anshika
anshika@LAPTOP-BLC9HHCI:~$ touch script.py
anshika@LAPTOP-BLC9HHCI:~$ ls -l script.py
-rw-r--r-- 1 anshika anshika 0 Mar 24 11:18 script.py
anshika@LAPTOP-BLC9HHCI:~$ chmod +x script.py
anshika@LAPTOP-BLC9HHCI:~$ ls -l script.py
-rwxr-xr-x 1 anshika anshika 0 Mar 24 11:18 script.py
anshika@LAPTOP-BLC9HHCI:~$
```

The permission structure also affects how Python interacts with system files and external resources. For instance, a script attempting to modify files in restricted directories such as /usr/bin will fail unless executed with administrative privileges. This ensures that sensitive parts of the system are protected from unauthorized changes. Additionally, permissions help maintain isolation between users, preventing one user's actions from affecting another's environment.

Understanding these mechanisms is essential for effectively working with Python in a Linux system. The combination of structured file organization, user-based access control, and permission enforcement ensures that Python can operate securely and efficiently in both individual and multi-user environments. These features highlight the strength of Linux as a platform for open-source software, where flexibility is balanced with strong system-level safeguards.

Execution, Management, and Update Mechanisms

Execution Model of Python in Linux Python operates in a Linux environment as an interpreted programming language rather than a
continuously running system service. This means that Python programs are executed on demand through the Python interpreter, typically using commands such as python3 script.py. Unlike services such as web servers or database systems that run in the background, Python executes scripts only when explicitly invoked by the user or another program. This execution model provides flexibility, allowing developers to run scripts for automation, data processing, and system tasks without requiring persistent system resources.

```
anshika@LAPTOP-BLC9HHCI:~$ echo "print('hello linux')" > script.py
anshika@LAPTOP-BLC9HHCI:~$ python3 script.py
hello linux
anshika@LAPTOP-BLC9HHCI:~$
```

In addition to direct execution, Python scripts can also be made executable within the Linux environment by assigning appropriate permissions and including a shebang line at the beginning of the script. This allows scripts to be executed similarly to standard system commands, further integrating Python into the Linux ecosystem. The ability to execute scripts in multiple ways reflects Python's adaptability and its role as both a scripting and general-purpose programming language.

Package Management and Dependency Handling The management of Python within a Linux system is closely tied to the operating system's package
management tools as well as Python-specific package managers. System-level installation and updates are typically handled through package managers such as apt, which ensure that Python and its core libraries are compatible with the overall system environment. These tools simplify software maintenance by automatically resolving dependencies and managing version control.

```
anshika@LAPTOP-BLC9HHCI:~$ pip3 --version
pip 24.0 from /usr/lib/python3/dist-packages/pip (python 3.12)
anshika@LAPTOP-BLC9HHCI:~$
```

In addition to system-level management, Python provides its own package management tool known as pip, which is used to install, update, and manage third-party libraries. This allows developers to extend Python's functionality by incorporating external modules tailored to specific applications. The separation between system-level and user-level package management helps maintain system stability while providing flexibility for development. Users can install libraries in isolated environments without

affecting the core system configuration, which is particularly important in multi-user or production environments.

Update Mechanisms and Continuous Development

The update process for Python in Linux systems reflects the collaborative nature of open-source software development. Updates, including security patches, bug fixes, and performance improvements, are developed by a global community and distributed through official repositories. These updates can be applied using commands such as sudo apt update and sudo apt upgrade, ensuring that the system remains secure and up to date.

```
anshika@LAPTOP-BLC9HHCI:~$ sudo apt update
Hit:1 http://security.ubuntu.com/ubuntu noble-security InRelease
Hit:2 http://archive.ubuntu.com/ubuntu noble InRelease
Hit:3 http://archive.ubuntu.com/ubuntu noble-updates InRelease
Hit:4 http://archive.ubuntu.com/ubuntu noble-backports InRelease
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
54 packages can be upgraded. Run 'apt list --upgradable' to see them.
anshika@LAPTOP-BLC9HHCI:~$
```

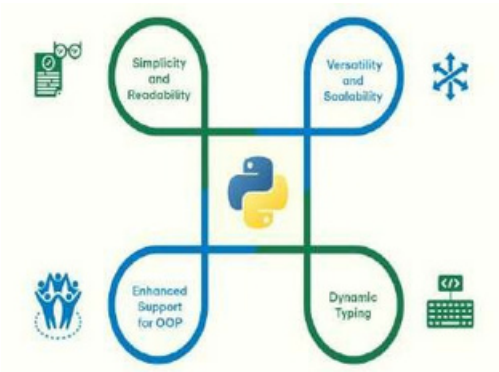
Unlike proprietary software, where updates are controlled by a single organization, Python evolves through community contributions and peer review. This model enables rapid identification of issues and continuous enhancement of the software. At the same time, the structured release process ensures stability and reliability, which is essential for both individual users and large-scale deployments. The combination of community-driven development and organized distribution mechanisms allows Python to maintain a balance between innovation and dependability.

Part – C:

Ecosystem and Role ofPythoninModern Software Systems

Python in the Modern Open Source Ecosystem The modern software ecosystem consists of a wide range of interconnected tools, frameworks, libraries, and platforms that support application development across multiple domains. Within this ecosystem, Python holds a central position due to its simplicity, versatility, and extensive library support.

Python is not limited to a single domain. It is widely used in web development, data science, artificial intelligence, automation, and system scripting. This broad applicability allows Python to act as a unifying technology that connects different layers of software development. One of the most important strengths of Python is its rich ecosystem of open-source libraries and frameworks. Tools such as Django and Flask are commonly used for web development, while libraries like NumPy, Pandas, and TensorFlow support data processing and machine learning tasks. These tools are continuously improved by a global developer community, ensuring that Python remains relevant and adaptable to new technological trends. Another important aspect is the strong community support and extensive documentation available for Python. Developers can easily access tutorials, forums, and open-source repositories, which reduces development time and effort. This ecosystem-driven approach encourages code reuse and collaboration, allowing developers to build complex systems efficiently without starting from scratch.



Python for Enterprise Software Development

Comparison with Traditional LAMP Stack Architecture The traditional LAMP stack consists of Linux, Apache HTTP Server, MySQL, and PHP. Each component has a specific role: Linux provides the operating system, Apache handles web requests, MySQL manages the database, and PHP is used for server-side scripting. While this architecture has been widely used and proven to be reliable, it is relatively rigid in terms of programming language choice. Python introduces more flexibility by replacing or complementing the scripting layer of the stack.

Instead of PHP, developers can use Python frameworks such as Django or Flask to build web applications. This leads to what is sometimes referred to as a **LAPP stack** (Linux, Apache, PostgreSQL/MySQL, Python). These frameworks follow modern design principles such as modularity, scalability, and separation of concerns, making them suitable for building structured and maintainable applications.

Additionally, Python-based systems are not restricted to specific technologies. Applications can be deployed using alternative web servers like Nginx and can work with databases such as PostgreSQL. This flexibility allows developers to design architectures based on project requirements rather than being limited to predefined stacks

Python's Role in Scalable and Modern Application Development

Python plays a significant role in modern software development, especially in building scalable and maintainable applications. Its clean syntax and modular design make it easier to write and manage large codebases. This is particularly important in environments where software systems need to evolve over time.

Python is widely used in cloud computing, automation, and data-driven applications, where rapid development and adaptability are essential. One of its key strengths is its ability to integrate with other technologies. It can serve as a backend language, interact with databases, process large datasets, and communicate with external services through APIs.

Another important advantage is the availability of frameworks and tools that support scalable development. These tools help with code organization, testing, and deployment, making it easier to manage complex applications. Although Python may not always provide the same performance as lower-level languages, its faster development time and ease of maintenance often make it a more practical choice in real-world scenarios.

Features that makes python popular

Part – D :

Open SourcevsProprietary Software

Dimension	Open Source (Python)	Proprietary Software (e.g., MATLAB)
Cost	Freely available with no licensing fees	Requires paid licenses, often expensive
Source Code	Fully open and accessible to users	Source code is closed and restricted
Security	Can be audited and improved by the community	Depends on vendor for security updates
Support and Reliability	Community support through forums and documentation	Official support from the company
Freedom to Modify	Users can modify and redistribute freely	Limited or no permission to modify
Control	Community-driven development	Controlled by a single organization

Comparison Between Open Source (Python) and Proprietary Software

Analysis and Discussion Open-source software such as Python provides significant advantages in terms of flexibility, transparency, and accessibility. Since the source code is openly available, developers are not restricted by predefined limitations and can adapt the software to suit specific requirements. This openness encourages collaboration across a global community, leading to continuous improvements and faster innovation. Additionally, the absence of licensing costs makes Python highly attractive for students, startups, and research institutions, where budget constraints are often a concern.

Another important aspect is security. Contrary to the common misconception that open-source software is less secure, the availability of source code allows experts worldwide to audit and identify vulnerabilities. When actively maintained, this can result in faster detection and patching of issues. However, this benefit depends on the strength of the community and the level of ongoing maintenance. On the other hand, proprietary software such as MATLAB offers a more structured and controlled environment. Since it is developed by a single organization, users benefit from standardized features, professional documentation, and dedicated technical support. This makes proprietary tools particularly useful in enterprise environments where reliability, accountability, and guaranteed

support are critical factors. However, these advantages come at the cost of reduced flexibility, as users cannot modify the source code or customize the software beyond the options provided by the vendor.

Furthermore, organizations using proprietary software are often dependent on the vendor for updates, bug fixes, and new features. This can lead to vendor lock-in, where switching to another platform becomes difficult due to compatibility and cost issues. In contrast, open-source software reduces such dependency, giving users more control over their technological choices.

Conclusion and Personal Verdict In my view, open-source software like Python represents a more adaptable and future-oriented approach to software development. Its flexibility, cost-effectiveness, and strong ecosystem make it suitable for a wide range of applications, from academic use to large-scale industrial systems. The ability to modify and extend the software provides a significant advantage in dynamic environments where requirements frequently change. While proprietary software offers benefits such as dedicated support and a stable, well-defined environment, these advantages are often outweighed by the limitations in flexibility and higher costs. For most modern use cases, especially those involving innovation and scalability, Python and other open-source technologies provide a more practical solution.

Therefore, I would recommend adopting open-source software like Python in most scenarios, particularly where customization, cost efficiency, and long-term sustainability are important considerations. The open-source model not only promotes collaboration but also empowers users to actively contribute to and shape the technology they use.

Part – E:

ImplementationUsingShell Scripting

This section presents thepracticalimplementation ofthe projectusing Linux shell scripting. A menu-driven command-line interface was developed to integrate multiple scripts into a single executable system. Each script performs a specific task such as system information retrieval, file operations, user interaction, process monitoring, and automation. This implementation demonstrates how open-source tools and Linux commands can be used to build functional and efficient system-level utilities.

Project Structure:

python-oss-audit/
--main.sh
--scripts/

-system_info.sh
-file_ops.sh
-user_interaction.sh
-process_check.sh
-automation.sh

The project follows a modular structure where each script performs a specific task. The main.sh file acts as the entry point and provides a menu-driven interface to execute the individual scripts.

1. Main Script (main.sh)

The main.sh script serves as the central controller of the project. It displays a menu-driven interface that allows the user to select different operations. Based on the user's input, the script uses a case statement o execute the corresponding shell script. The loop ensures that the menu continues to run

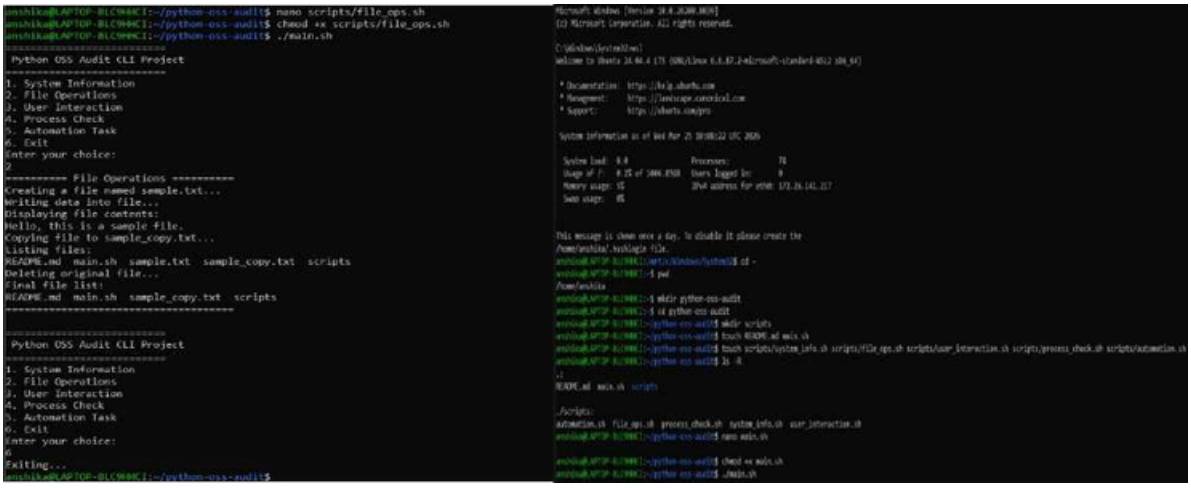


Figure 1: Menu-driven CLIinterface of the project

2. Script 1: System Information

This script displays system-level information such as the logged-in user, current working directory, date and time, system uptime, and disk usage. It also shows the Python version and its installation path.

```
anshika@LAPTOP-BLC9HHCI:~/python-oss-audit$ nano scripts/system_info.sh
anshika@LAPTOP-BLC9HHCI:~/python-oss-audit$ chmod +x scripts/system_info.sh
anshika@LAPTOP-BLC9HHCI:~/python-oss-audit$ ./main.sh
==========================
 Python OSS Audit CLI Project
==========================
1. System Information
2. File Operations
3. User Interaction
4. Process Check
5. Automation Task
6. Exit
Enter your choice:
1
```

```
========== System Information ==========
Logged-in User:
anshika
Current Working Directory:
/home/anshika/python-oss-audit
Date and Time:
Wed Mar 25 10:39:49 UTC 2026
System Uptime:
 10:39:49 up 31 min,  1 user,  load average: 0.00, 0.00, 0.00
---------------------------------------
Python Version:
Python 3.12.3
Python Executable Path:
/usr/bin/python3
---------------------------------------
Disk Usage:
Filesystem      Size  Used Avail Use% Mounted on
none            3.8G     0  3.8G   0% /usr/lib/modules/6.6.87.2-microsoft-standard-WSL2
none            3.8G  4.0K  3.8G   1% /mnt/wsl
drivers         173G  145G   29G  84% /usr/lib/wsl/drivers
/dev/sdd       1007G  1.9G  954G   1% /
=======================================

==========================
 Python OSS Audit CLI Project
==========================
1. System Information
2. File Operations
3. User Interaction
4. Process Check
5. Automation Task
6. Exit
Enter your choice:
6
Exiting...
anshika@LAPTOP-BLC9HHCI:~/python-oss-audit$
```

Figure 2: Output showing system and Python information

3. Script 2: File Operations

This script performs file handling operations including creating a file, writing content, displaying the file, copying it, and deleting it. Commands like touch, echo, cat, cp, and rm are used.

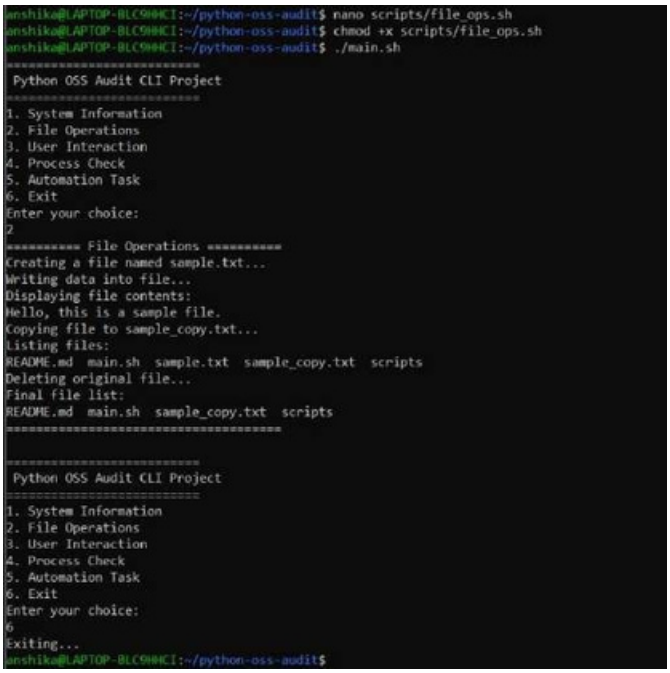


Figure 3: File creation and file operations

4. Script 3: User Interaction

This script takes input from the user and performs a simple calculation. It demonstrates how shell scripts can interact with users and process input.

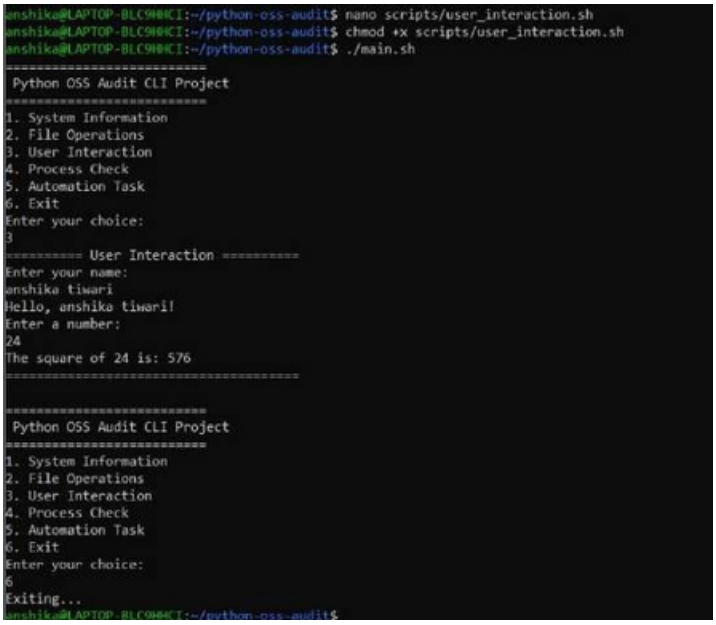


Figure 4: User input and output processing

5. Script 4: Process Check

This script displays running processes in the system and filters processes related to Python using p and grep commands

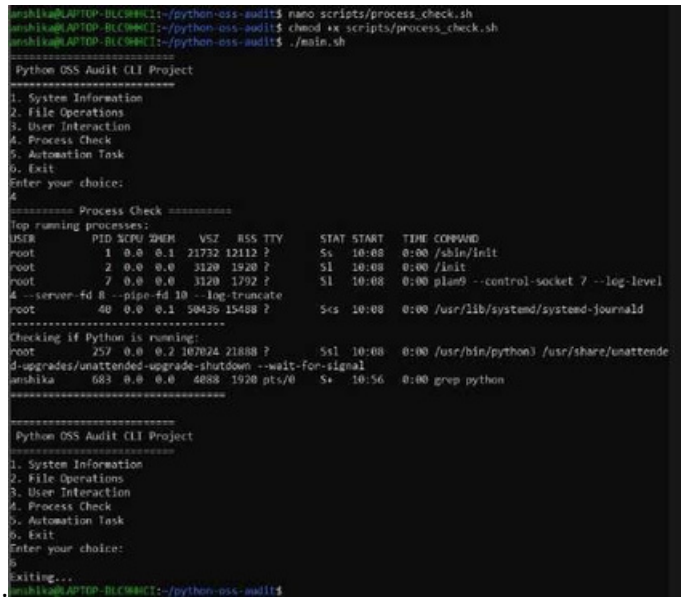


Figure 5: Display of running processes

6. Script 5: Automation Task

This script creates a log file and stores system information such as date, user, directory, and Python version. It demonstrates automation using shell scripting.

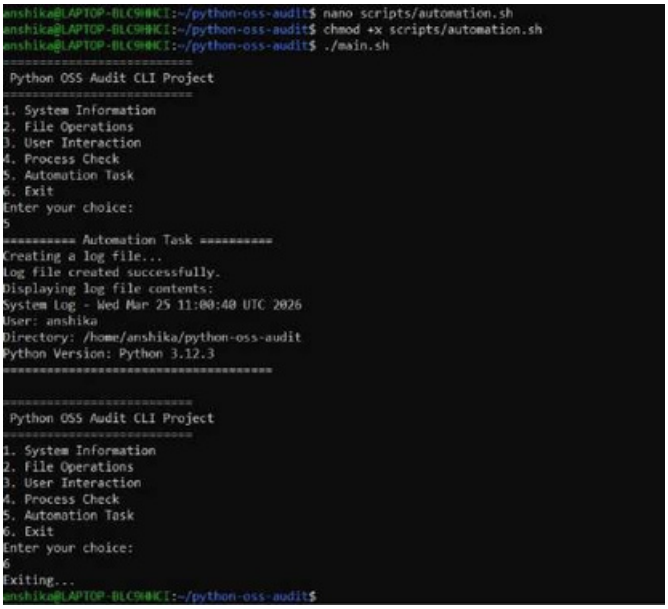


Figure 6: Automated log file generation

Conclusion

The study of Python as an open-source software highlights its significance as a versatile and widely adopted programming language within modern computing environments. Its integration into Linux systems, extensive ecosystem of libraries and frameworks, and strong community support demonstrate its role as both a development tool and a system-level utility. The structured organization of Python within the Linux file system, along with its interaction with users and permission mechanisms, reflects the efficiency and flexibility of open-source operating systems.

In addition to its technical capabilities, Python's position within the broader software ecosystem illustrates a shift toward modular, collaborative, and scalable development practices. Its compatibility with various technologies and its ability to function across multiple domains make it a central component in contemporary software solutions. The comparison with traditional architectures further emphasizes its adaptability and relevance in evolving technological landscapes. While Python presents certain limitations in terms of performance and resource management, its advantages in readability, rapid development, and community-driven improvement often outweigh these challenges in practical applications. The balance between simplicity and functionality enables it to remain accessible to beginners while still being powerful enough for complex projects.

Overall, Python represents an effective model of open-source software, where continuous development, community participation, and practical usability combine to create a reliable and sustainable technology. Its continued growth and widespread adoption indicate its lasting importance in the field of software development.